

Printout

Thursday, March 5, 2026 12:14 AM

✓ Lab - 1 | 22 January 2026

Meta Pitts Neuron (AND & OR Gates)

```
import numpy as np
def mp_neuron(x,w,theta):
    return 1 if np.dot(x, w) >= theta else 0

print("-----AND GATE-----")

# AND Gate
w_and = np.array ([1,1])
theta_and = 2
for x in [(0,0), (0,1), (1,0), (1,1)]:
    print(x, mp_neuron(np.array(x), w_and, theta_and))

print("-----OR GATE-----")

# OR Gate
w_or = np.array ([1,1])
theta_or = 1
for x in [(0,0), (0,1), (1,0), (1,1)]:
    print(x, mp_neuron(np.array(x), w_or, theta_or))

print("+-----+")
print("Gaurav Mehra, 1/23/SET/BCS/423")
print("22 January 2026")
```

```
-----AND GATE-----
(0, 0) 0
(0, 1) 0
(1, 0) 0
(1, 1) 1
-----OR GATE-----
(0, 0) 0
(0, 1) 1
(1, 0) 1
(1, 1) 1
+-----+
Gaurav Mehra, 1/23/SET/BCS/423
22 January 2026
```

AND & OR GATE(3-inputs)

```
import numpy as np
```

```

def mp_neuron(x,w,theta):
    return 1 if np.dot(x, w) >= theta else 0

print("-----AND GATE(3-inputs)-----")

w_and = np.array ([1,1,1])
theta_and = 3
for x in [(0,0,0), (0,0,1), (0,1,0), (0,1,1), (1,0,0), (1,0,1), (1,1,0), (1,
    print(x, mp_neuron(np.array(x), w_and, theta_and))

print("-----OR GATE(3-inputs)-----")

w_or = np.array ([1,1,1])
theta_or = 1
for x in [(0,0,0), (0,0,1), (0,1,0), (0,1,1), (1,0,0), (1,0,1), (1,1,0), (1,
    print(x, mp_neuron(np.array(x), w_or, theta_or))

print("+-----+")
print("Gaurav Mehra, 1/23/SET/BCS/423")
print("22 January 2026")

```

```

-----AND GATE(3-inputs)-----
(0, 0, 0) 0
(0, 0, 1) 0
(0, 1, 0) 0
(0, 1, 1) 0
(1, 0, 0) 0
(1, 0, 1) 0
(1, 1, 0) 0
(1, 1, 1) 1
-----OR GATE(3-inputs)-----
(0, 0, 0) 0
(0, 0, 1) 1
(0, 1, 0) 1
(0, 1, 1) 1
(1, 0, 0) 1
(1, 0, 1) 1
(1, 1, 0) 1
(1, 1, 1) 1
+-----+
Gaurav Mehra, 1/23/SET/BCS/423
22 January 2026

```

Artificial Neuron

```

import numpy as np

def art_neuron(x, w, b, activation='step'):
    z = np.dot(x, w) + b

```

```

    if activation == 'step':
        return 1 if z >= 0 else 0
    elif activation == 'sigmoid':
        return 1 / (1 + np.exp(-z))
    elif activation == 'relu':
        return max(0, z)
    else:
        return "Invalid Activation Function"

x = np.array([1,2])
w = np.array([0.5,1.5])
b = -1

print("-----")
print("Step Output      :", art_neuron(x, w, b, activation='step'))
print("-----")
print("Sigmoid Output   :", art_neuron(x, w, b, activation='sigmoid'))
print("-----")
print("ReLu Output      :", art_neuron(x, w, b, activation='relu'))

print("+-----+")
print("Gaurav Mehra, 1/23/SET/BCS/423")
print("22 January 2026")

```

```

-----
Step Output      : 1
-----
Sigmoid Output   : 0.9241418199787566
-----
ReLu Output      : 2.5
+-----+
Gaurav Mehra, 1/23/SET/BCS/423
22 January 2026

```

✓ Lab - 2 | 29 January 2026

Perceptron learning Algorithm for Implementing AND Logic Gate

```

import numpy as np
# for AND gate (Input/Output)
x_list = np.array([[0,0],[0,1],[1,0],[1,1]])
y_list = np.array([0,0,0,1])
epochs = 4
w,b = np.zeros(2),0

for _ in range(epochs):
    for x,y in zip(x_list,y_list):

```

```

    z = np.dot(x,w) + b
    ### condition area for y_pred
    y_pred = 1 if z >= 0 else 0
    E = y - y_pred
    print("error : ",E,"<- at epoch",_)
    w += E*x
    b += E
    print(w,b)
print("+-+-----+")
print("Gaurav Mehra, 1/23/SET/BCS/423")
print("22 January 2026")

```

```

error :  -1 <- at epoch 0
[0. 0.] -1
error :  0 <- at epoch 0
[0. 0.] -1
error :  0 <- at epoch 0
[0. 0.] -1
error :  1 <- at epoch 0
[1. 1.] 0
error :  -1 <- at epoch 1
[1. 1.] -1
error :  -1 <- at epoch 1
[1. 0.] -2
error :  0 <- at epoch 1
[1. 0.] -2
error :  1 <- at epoch 1
[2. 1.] -1
error :  0 <- at epoch 2
[2. 1.] -1
error :  -1 <- at epoch 2
[2. 0.] -2
error :  -1 <- at epoch 2
[1. 0.] -3
error :  1 <- at epoch 2
[2. 1.] -2
error :  0 <- at epoch 3
[2. 1.] -2
error :  0 <- at epoch 3
[2. 1.] -2
error :  -1 <- at epoch 3
[1. 1.] -3
error :  1 <- at epoch 3
[2. 2.] -2
+-+-----+
Gaurav Mehra, 1/23/SET/BCS/423
22 January 2026

```

Tasks to be performed

1. Change the epochs and observe the output
2. Change the random weights and bias and observe the convergence of the network

network.

3. Add logic to stop the training when the error is 'zero' for all the inputs.
4. Repeat the same process for OR gate
5. Try the process for XOR gate and show that the network will not stabilize (Converge).

Tasks in Order :

1) Change the epochs and observe the output

```
import numpy as np
# for AND gate (Input/Output)
x_list = np.array([[0,0],[0,1],[1,0],[1,1]])
y_list = np.array([0,0,0,1])
epochs = 10
w,b = np.zeros(2),0

for _ in range(epochs):
    for x,y in zip(x_list,y_list):
        z = np.dot(x,w) + b
        ### condition area for y_pred
        y_pred = 1 if z >= 0 else 0
        E = y - y_pred
        print("error : ",E,"<- at epoch",_)
        w += E*x
        b += E
    print(w,b)
print("+-----+")
print("Gaurav Mehra, 1/23/SET/BCS/423")
print("22 January 2026")
```

```
error :  -1 <- at epoch 0
[0. 0.] -1
error :  0 <- at epoch 0
[0. 0.] -1
error :  0 <- at epoch 0
[0. 0.] -1
error :  1 <- at epoch 0
[1. 1.] 0
error :  -1 <- at epoch 1
[1. 1.] -1
error :  -1 <- at epoch 1
[1. 0.] -2
error :  0 <- at epoch 1
[1. 0.] -2
error :  1 <- at epoch 1
[2. 1.] -1
error :  0 <- at epoch 2
[2. 1.] -1
```

```

error : -1 <- at epoch 2
[2. 0.] -2
error : -1 <- at epoch 2
[1. 0.] -3
error : 1 <- at epoch 2
[2. 1.] -2
error : 0 <- at epoch 3
[2. 1.] -2
error : 0 <- at epoch 3
[2. 1.] -2
error : -1 <- at epoch 3
[1. 1.] -3
error : 1 <- at epoch 3
[2. 2.] -2
error : 0 <- at epoch 4
[2. 2.] -2
error : -1 <- at epoch 4
[2. 1.] -3
error : 0 <- at epoch 4
[2. 1.] -3
error : 0 <- at epoch 4
[2. 1.] -3
error : 0 <- at epoch 5
[2. 1.] -3
error : 0 <- at epoch 5
[2. 1.] -3
error : 0 <- at epoch 5
[2. 1.] -3
error : 0 <- at epoch 5
[2. 1.] -3
error : 0 <- at epoch 6
[2. 1.] -3
error : 0 <- at epoch 6
[2. 1.] -3
error : 0 <- at epoch 6
[2. 1.] -3
error : 0 <- at epoch 6
[2. 1.] -3
error : 0 <- at epoch 7
[2. 1.] -3

```

2) Change the random weights and bias and observe the convergence of the network.

```

import numpy as np
# for AND gate (Input/Output)
x_list = np.array([[0,0],[0,1],[1,0],[1,1]])
y_list = np.array([0,0,0,1])
epochs = 10
w,b = np.array([2,5]),0

for _ in range(epochs):
    for i in range(len(x_list)):

```

```

for x,y in zip(x_list,y_list):
    z = np.dot(x,w) + b
    ### condition area for y_pred
    y_pred = 1 if z >= 0 else 0
    E = y - y_pred
    print("error : ",E,"<- at epoch",_)
    w += E*x
    b += E
    print(w,b)
print("+-----+")
print("Gaurav Mehra, 1/23/SET/BCS/423")
print("22 January 2026")

```

```

error : -1 <- at epoch 0
[2 5] -1
error : -1 <- at epoch 0
[2 4] -2
error : -1 <- at epoch 0
[1 4] -3
error : 0 <- at epoch 0
[1 4] -3
error : 0 <- at epoch 1
[1 4] -3
error : -1 <- at epoch 1
[1 3] -4
error : 0 <- at epoch 1
[1 3] -4
error : 0 <- at epoch 1
[1 3] -4
error : 0 <- at epoch 2
[1 3] -4
error : 0 <- at epoch 2
[1 3] -4
error : 0 <- at epoch 2
[1 3] -4
error : 0 <- at epoch 2
[1 3] -4
error : 0 <- at epoch 3
[1 3] -4
error : 0 <- at epoch 3
[1 3] -4
error : 0 <- at epoch 3
[1 3] -4
error : 0 <- at epoch 4
[1 3] -4
error : 0 <- at epoch 4
[1 3] -4
error : 0 <- at epoch 4
[1 3] -4
error : 0 <- at epoch 4
[1 3] -4
error : 0 <- at epoch 5
[1 3] -4

```



```

[1 3] -4
error : 0 <- at epoch 5
[1 3] -4
error : 0 <- at epoch 5
[1 3] -4
error : 0 <- at epoch 5
[1 3] -4
error : 0 <- at epoch 6
[1 3] -4
error : 0 <- at epoch 6
[1 3] -4
error : 0 <- at epoch 6
[1 3] -4
error : 0 <- at epoch 6
[1 3] -4
error : 0 <- at epoch 7
[1 3] -4

```

3) Add logic to stop the training when the error is 'zero' for all the inputs.

```

import numpy as np
# for AND gate (Input/Output)
x_list = np.array([[0,0],[0,1],[1,0],[1,1]])
y_list = np.array([0,0,0,1])
epochs = 10 # Increased epochs to ensure convergence without fixed iteration
w,b = np.zeros(2),0

for _ in range(epochs):
    errors_in_epoch = False # Flag to track errors in the current epoch
    for x,y in zip(x_list,y_list):
        z = np.dot(x,w) + b
        y_pred = 1 if z >= 0 else 0
        E = y - y_pred
        print("error : ",E,"is at epoch",_)
        if E != 0:
            w += E*x
            b += E
            errors_in_epoch = True
    print(w,b)
    if not errors_in_epoch:
        print(f"Training stopped at epoch {_} as error has become zero for all t
        print("+-----+")
        print("Gaurav Mehra, 1/23/SET/BCS/423")
        print("22 January 2026")
        break

error : -1 is at epoch 0
[0. 0.] -1
error : 0 is at epoch 0
[0. 0.] -1
error : 0 is at epoch 0
[0. 0.] -1

```

```

error : 1 is at epoch 0
[1. 1.] 0
error : -1 is at epoch 1
[1. 1.] -1
error : -1 is at epoch 1
[1. 0.] -2
error : 0 is at epoch 1
[1. 0.] -2
error : 1 is at epoch 1
[2. 1.] -1
error : 0 is at epoch 2
[2. 1.] -1
error : -1 is at epoch 2
[2. 0.] -2
error : -1 is at epoch 2
[1. 0.] -3
error : 1 is at epoch 2
[2. 1.] -2
error : 0 is at epoch 3
[2. 1.] -2
error : 0 is at epoch 3
[2. 1.] -2
error : -1 is at epoch 3
[1. 1.] -3
error : 1 is at epoch 3
[2. 2.] -2
error : 0 is at epoch 4
[2. 2.] -2
error : -1 is at epoch 4
[2. 1.] -3
error : 0 is at epoch 4
[2. 1.] -3
error : 0 is at epoch 4
[2. 1.] -3
error : 0 is at epoch 5
[2. 1.] -3
error : 0 is at epoch 5
[2. 1.] -3
error : 0 is at epoch 5
[2. 1.] -3
error : 0 is at epoch 5
[2. 1.] -3
Training stopped at epoch 5 as error has become zero for all the following in
+-----+
Gaurav Mehra, 1/23/SET/BCS/423
22 January 2026

```

4) Repeat the same process for OR gate

```

import numpy as np

x_list = np.array([[0,0],[0,1],[1,0],[1,1]])
y_list = np.array([0,1,1,1])

```

```

epochs = 10
w,b = np.zeros(2),0

print("-----OR GATE Perceptron Learning-----")
for _ in range(epochs):
    errors_in_epoch = False
    for x,y in zip(x_list,y_list):
        z = np.dot(x,w) + b
        y_pred = 1 if z >= 0 else 0
        E = y - y_pred
        print(f"Error: {E}, Epoch: {_}")
        if E != 0:
            w += E*x
            b += E
            errors_in_epoch = True
    print(f"Updated weights: {w}, Updated bias: {b}")
    if not errors_in_epoch:
        print(f"Training stopped at epoch {_} as error has become zero for all i
        break

print("+-----+")
print("Gaurav Mehra, 1/23/SET/BCS/423")
print("22 January 2026")

```

```

-----OR GATE Perceptron Learning-----
Error: -1, Epoch: 0
Updated weights: [0. 0.], Updated bias: -1
Error: 1, Epoch: 0
Updated weights: [0. 1.], Updated bias: 0
Error: 0, Epoch: 0
Updated weights: [0. 1.], Updated bias: 0
Error: 0, Epoch: 0
Updated weights: [0. 1.], Updated bias: 0
Error: -1, Epoch: 1
Updated weights: [0. 1.], Updated bias: -1
Error: 0, Epoch: 1
Updated weights: [0. 1.], Updated bias: -1
Error: 1, Epoch: 1
Updated weights: [1. 1.], Updated bias: 0
Error: 0, Epoch: 1
Updated weights: [1. 1.], Updated bias: 0
Error: -1, Epoch: 2
Updated weights: [1. 1.], Updated bias: -1
Error: 0, Epoch: 2
Updated weights: [1. 1.], Updated bias: -1
Error: 0, Epoch: 2
Updated weights: [1. 1.], Updated bias: -1
Error: 0, Epoch: 2
Updated weights: [1. 1.], Updated bias: -1
Error: 0, Epoch: 3
Updated weights: [1. 1.], Updated bias: -1
Error: 0, Epoch: 3
Updated weights: [1. 1.], Updated bias: -1
Error: 0, Epoch: 3

```

```

error: 0, epoch: 3
Updated weights: [1. 1.], Updated bias: -1
Error: 0, Epoch: 3
Updated weights: [1. 1.], Updated bias: -1
Training stopped at epoch 3 as error has become zero for all inputs.
+-----+
Gaurav Mehra, 1/23/SET/BCS/423
22 January 2026

```

5) Try the process for XOR gate and show that the network will not stabilize (Converge).

```

import numpy as np

x_list = np.array([[0,0],[0,1],[1,0],[1,1]])
y_list = np.array([0,1,1,0])
epochs = 10
w,b = np.zeros(2),0

print("-----OR GATE Perceptron Learning-----")
for _ in range(epochs):
    errors_in_epoch = False
    for x,y in zip(x_list,y_list):
        z = np.dot(x,w) + b
        y_pred = 1 if z >= 0 else 0
        E = y - y_pred
        print(f"Error: {E}, Epoch: {_}")
        if E != 0:
            w += E*x
            b += E
            errors_in_epoch = True
    print(f"Updated weights: {w}, Updated bias: {b}")
    if not errors_in_epoch:
        print(f"Training stopped at epoch {_} as error has become zero for all i
        break

print("+-----+")
print("Gaurav Mehra, 1/23/SET/BCS/423")
print("22 January 2026")

-----OR GATE Perceptron Learning-----
Error: -1, Epoch: 0
Updated weights: [0. 0.], Updated bias: -1
Error: 1, Epoch: 0
Updated weights: [0. 1.], Updated bias: 0
Error: 0, Epoch: 0
Updated weights: [0. 1.], Updated bias: 0
Error: -1, Epoch: 0
Updated weights: [-1.  0.], Updated bias: -1
Error: 0, Epoch: 1
Updated weights: [-1.  0.], Updated bias: -1

```

```

Error: 1, Epoch: 1
Updated weights: [-1.  1.], Updated bias: 0
Error: 1, Epoch: 1
Updated weights: [0.  1.], Updated bias: 1
Error: -1, Epoch: 1
Updated weights: [-1.  0.], Updated bias: 0
Error: -1, Epoch: 2
Updated weights: [-1.  0.], Updated bias: -1
Error: 1, Epoch: 2
Updated weights: [-1.  1.], Updated bias: 0
Error: 1, Epoch: 2
Updated weights: [0.  1.], Updated bias: 1
Error: -1, Epoch: 2
Updated weights: [-1.  0.], Updated bias: 0
Error: -1, Epoch: 3
Updated weights: [-1.  0.], Updated bias: -1
Error: 1, Epoch: 3
Updated weights: [-1.  1.], Updated bias: 0
Error: 1, Epoch: 3
Updated weights: [0.  1.], Updated bias: 1
Error: -1, Epoch: 3
Updated weights: [-1.  0.], Updated bias: 0
Error: -1, Epoch: 4
Updated weights: [-1.  0.], Updated bias: -1
Error: 1, Epoch: 4
Updated weights: [-1.  1.], Updated bias: 0
Error: 1, Epoch: 4
Updated weights: [0.  1.], Updated bias: 1
Error: -1, Epoch: 4
Updated weights: [-1.  0.], Updated bias: 0
Error: -1, Epoch: 5
Updated weights: [-1.  0.], Updated bias: -1
Error: 1, Epoch: 5
Updated weights: [-1.  1.], Updated bias: 0
Error: 1, Epoch: 5
Updated weights: [0.  1.], Updated bias: 1
Error: -1, Epoch: 5
Updated weights: [-1.  0.], Updated bias: 0
Error: -1, Epoch: 6
Updated weights: [-1.  0.], Updated bias: -1
Error: 1, Epoch: 6
Updated weights: [-1.  1.], Updated bias: 0
Error: 1, Epoch: 6
Updated weights: [0.  1.], Updated bias: 1
Error: -1, Epoch: 6
Updated weights: [-1.  0.], Updated bias: 0
Error: -1, Epoch: 7

```

✓ LAB -3 5 Feb

```

import numpy as np
import matplotlib.pyplot as plt

```

```

# ----- Input & Target -----
x1, x2 = 0.6, 0.1
y = 1

# ----- Initial Parameters -----
w1, w2, b = 0.2, -0.3, 0.4
lr = 0.1

# ----- Sigmoid Function -----
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# ----- Forward Pass -----
z = w1 * x1 + w2 * x2 + b
y_pred = sigmoid(z)

# Error BEFORE update
error = abs(y - y_pred)

# ----- Backpropagation -----
dL_dy_pred = y_pred - y
dy_pred_dz = y_pred * (1 - y_pred)
dL_dz = dL_dy_pred * dy_pred_dz

dL_dw1 = dL_dz * x1
dL_dw2 = dL_dz * x2
dL_db = dL_dz

# ----- Parameter Update -----
w1 -= lr * dL_dw1
w2 -= lr * dL_dw2
b -= lr * dL_db

print("----- Updated Parameters -----")
print(f"w1: {w1:.4f}, w2: {w2:.4f}, b: {b:.4f}")
print(f"Prediction Error (before update): {error:.4f}")
print("-----")

# ----- Visualization -----
labels = ['Weight 1 (w1)', 'Weight 2 (w2)', 'Bias (b)', 'Error']
values = [w1, w2, b, error]
colors = ['royalblue', 'tomato', 'seagreen', 'gold'] # Different colors

plt.figure(figsize=(8, 5))
bars = plt.bar(labels, values, color=colors)

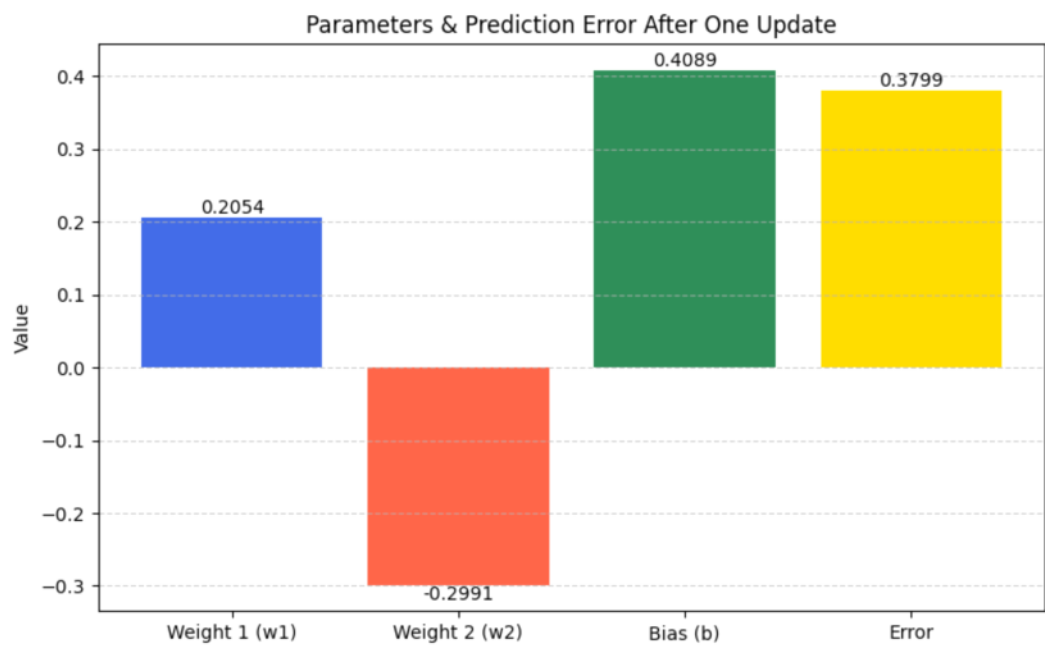
plt.title('Parameters & Prediction Error After One Update')
plt.ylabel('Value')
plt.grid(axis='y', linestyle='--', alpha=0.7)

```

```
# Add value labels on bars
for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, height,
             f'{height:.4f}', ha='center',
             va='bottom' if height >= 0 else 'top')

plt.tight_layout()
plt.show()
```

```
----- Updated Parameters -----
w1: 0.2054, w2: -0.2991, b: 0.4089
Prediction Error (before update): 0.3799
-----
```



```
import numpy as np
import matplotlib.pyplot as plt

X = np.array([
    [0,0],
    [0,1],
    [1,0],
    [1,1]
```

```

])
y = np.array([0,0,0,1])

learning_rates = [0.01, 0.1, 0.5, 0.8]
epochs = 1000

def sigmoid(z):
    return 1/(1+np.exp(-z))

plt.figure(figsize=(10,6))

for lr in learning_rates:

    w = np.array([0.1, 0.2], dtype=float)
    b = 0.3

    errors_per_epoch = []

    for epoch in range(epochs):
        total_absolute_error = 0

        for i in range(len(X)):
            z = np.dot(X[i], w) + b
            y_pred = sigmoid(z)

            total_absolute_error += abs(y[i] - y_pred)

            dz = y_pred - y[i]
            dw = dz * X[i]
            db = dz

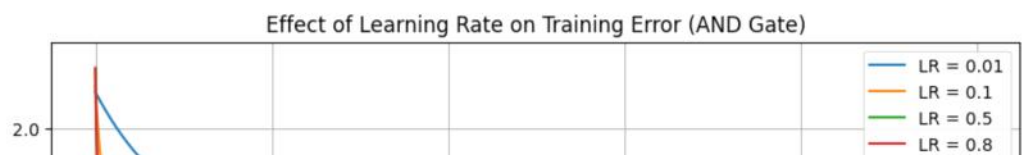
            w -= lr * dw
            b -= lr * db

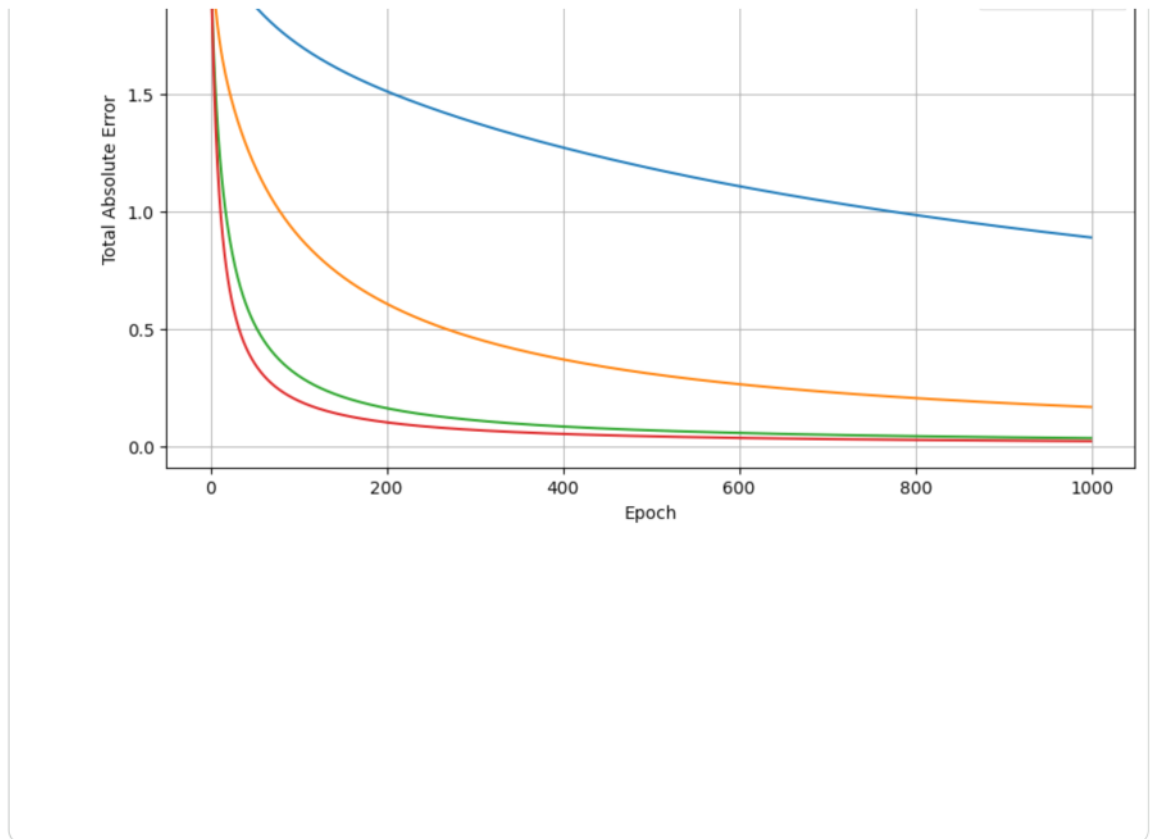
        errors_per_epoch.append(total_absolute_error)

    plt.plot(errors_per_epoch, label=f"LR = {lr}")

plt.title("Effect of Learning Rate on Training Error (AND Gate)")
plt.xlabel("Epoch")
plt.ylabel("Total Absolute Error")
plt.legend()
plt.grid(True)
plt.show()

```





✓ Lab- 4 | 12 Feb

Binary Search

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import accuracy_score, precision_score, recall_score, f

import seaborn as sns
import matplotlib.pyplot as plt
```

```
file_path = "/spam_or_not_spam.xlsx"
df = pd.read_excel(file_path)

print(df.head())
print(df.info())
```

```

                                email  label
0  date wed NUMBER aug NUMBER NUMBER NUMBER NUMB...    0.0
1  martin a posted tassos papadopoulos the greek ...    0.0
2  man threatens explosion in moscow thursday aug...    0.0
3  klez the virus that won t die already the most...    0.0
4  in adding cream to spaghetti carbonara which ...    0.0
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3016 entries, 0 to 3015
Data columns (total 2 columns):
#   Column  Non-Null Count  Dtype
---  ---
0  email    3015 non-null    object
1  label    3000 non-null    float64
dtypes: float64(1), object(1)
memory usage: 47.3+ KB
None

```

```

df = df.dropna(subset=["email", "label"])

X = df["email"]
y = df["label"]

print("Dataset shape:", df.shape)

```

Dataset shape: (2999, 2)

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

```

```

vectorizer = TfidfVectorizer(
    max_features=5000,
    stop_words='english',
    ngram_range=(1,2)
)

X_train = vectorizer.fit_transform(X_train).toarray()
X_test = vectorizer.transform(X_test).toarray()

print("X_train shape:", X_train.shape)

```

X_train shape: (2399, 5000)

```
y_train = np.array(y_train).reshape(-1,1)
y_test = np.array(y_test).reshape(-1,1)
```

```
input_size = X_train.shape[1]

np.random.seed(42)

W = np.random.randn(input_size, 1) * 0.01
b = np.zeros((1,1))

learning_rate = 0.1
epochs = 500
```

```
def sigmoid(Z):
    return 1 / (1 + np.exp(-Z))
```

```
def forward(X):
    Z = np.dot(X, W) + b
    A = sigmoid(Z)
    return A
```

```
def compute_loss(A, y):
    m = y.shape[0]
    epsilon = 1e-8

    loss = - (1/m) * np.sum(
        y*np.log(A+epsilon) +
        (1-y)*np.log(1-A+epsilon)
    )
    return loss
```

```
def backward(X, y, A):
    m = X.shape[0]

    dZ = A - y
    dW = (1/m) * np.dot(X.T, dZ)
    db = (1/m) * np.sum(dZ)

    return dW, db
```

```

loss_history = []
accuracy_history = []

for i in range(epochs):

    # Forward
    A = forward(X_train)

    # Loss
    loss = compute_loss(A, y_train)
    loss_history.append(loss)

    # Accuracy on training set
    predictions = (A >= 0.5).astype(int)
    train_acc = np.mean(predictions == y_train)
    accuracy_history.append(train_acc)

    # Backward
    dW, db = backward(X_train, y_train, A)

    # Update
    W -= learning_rate * dW
    b -= learning_rate * db

    if i % 100 == 0:
        print(f"Epoch {i}, Loss: {loss}, Accuracy: {train_acc}")

```

```

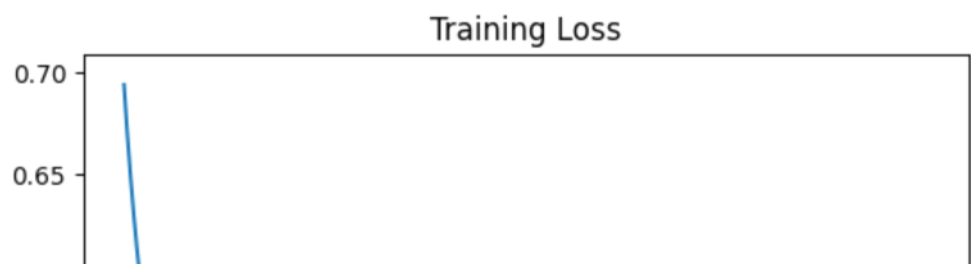
Epoch 0, Loss: 0.6938855908528171, Accuracy: 0.44018340975406417
Epoch 100, Loss: 0.441471697541632, Accuracy: 0.8336807002917882
Epoch 200, Loss: 0.426923304626026, Accuracy: 0.8336807002917882
Epoch 300, Loss: 0.417202053348458, Accuracy: 0.8336807002917882
Epoch 400, Loss: 0.4080324896469931, Accuracy: 0.8336807002917882

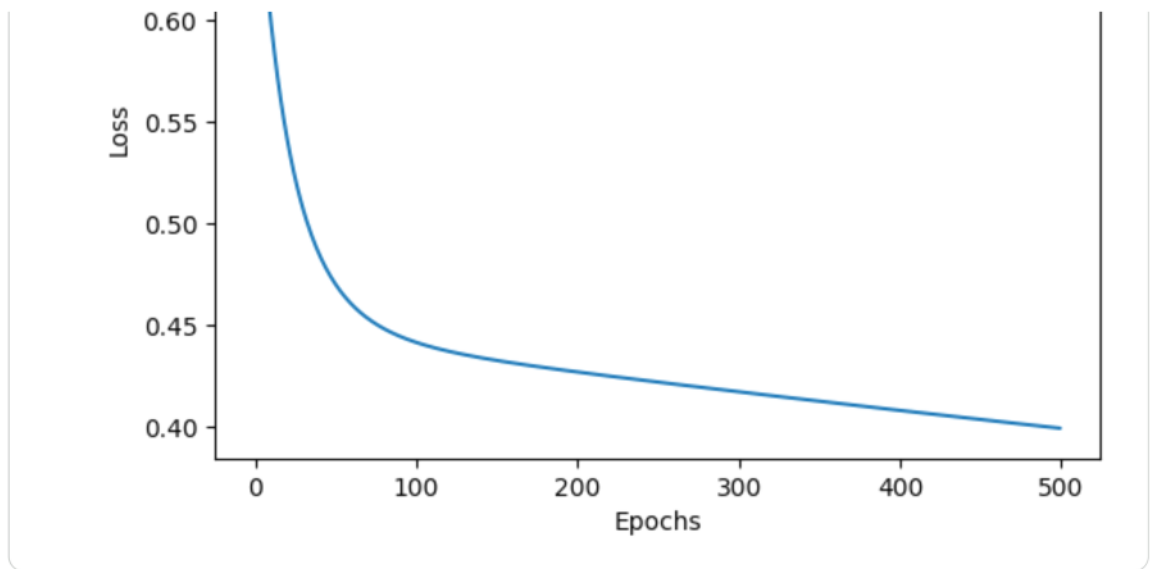
```

```

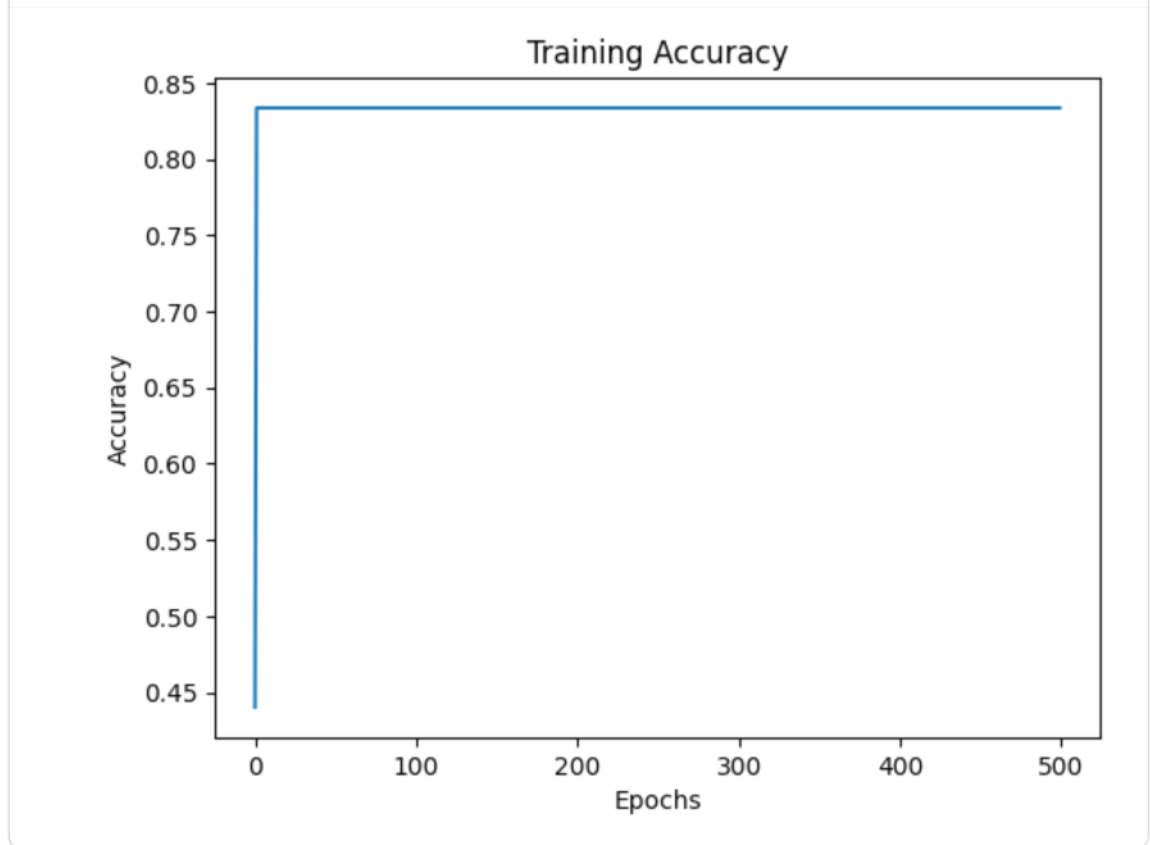
plt.figure()
plt.plot(loss_history)
plt.title("Training Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.show()

```





```
plt.figure()
plt.plot(accuracy_history)
plt.title("Training Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.show()
```



```
def predict(X, threshold=0.3):
    A = forward(X)
    return (A >= threshold).astype(int)
```

```
y_pred = predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)
```

```
Accuracy: 0.835
Precision: 1.0
Recall: 0.01
F1 Score: 0.019801980198019802
```

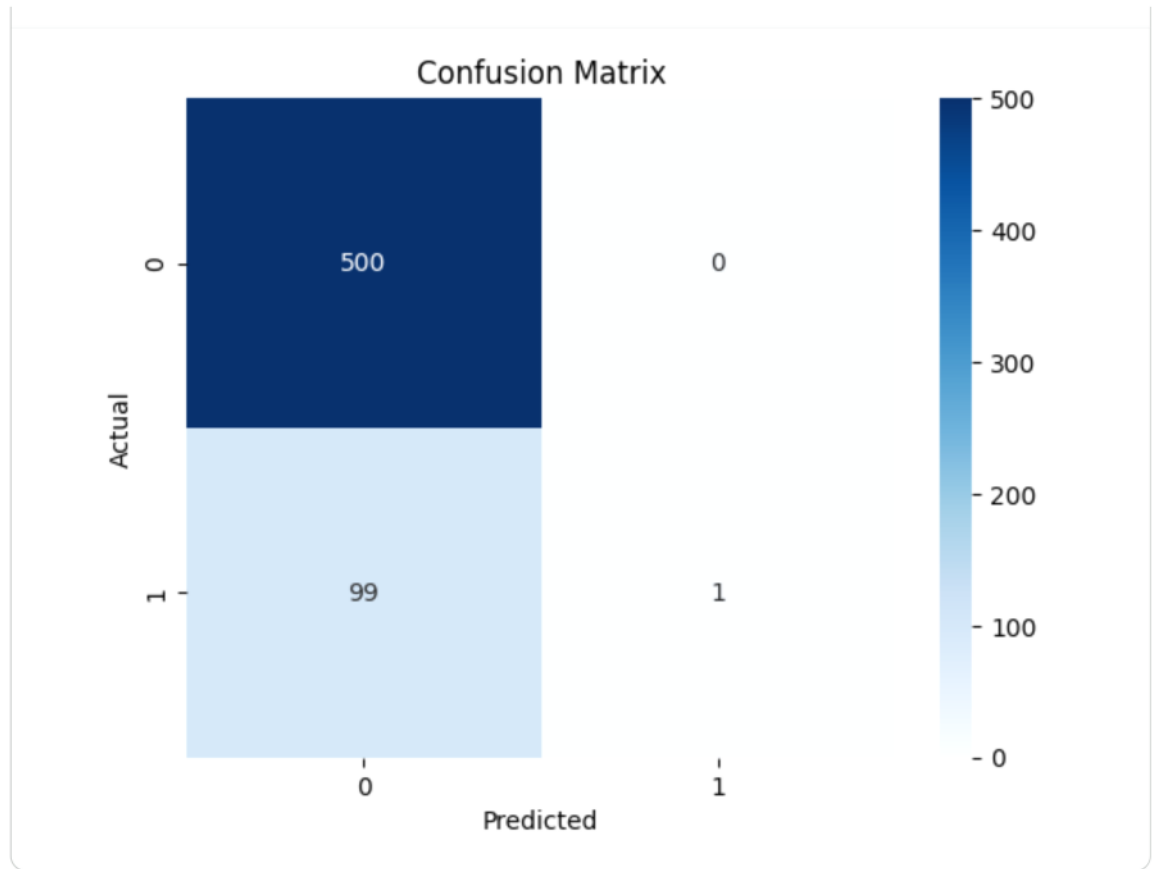
```
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))
```

Classification Report:

	precision	recall	f1-score	support
0.0	0.83	1.00	0.91	500
1.0	1.00	0.01	0.02	100
accuracy			0.83	600
macro avg	0.92	0.51	0.46	600
weighted avg	0.86	0.83	0.76	600

```
cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



✓ Lab - 5 | 19 Feb

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

y = np.array([0, 1, 1, 0])

model = Sequential()

model.add(Dense(4, input_dim=2, activation='relu'))

model.add(Dense(1, activation='sigmoid'))

model.compile(loss='binary_crossentropy',
```

```

optimizer='adam',
metrics=['accuracy'])

model.fit(X, y, epochs=500, verbose=0)

predictions = model.predict(X)

print("\nPredictions:")
for i in range(len(X)):
    print(f"Input: {X[i]} -> Predicted: {round(predictions[i][0])}")

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: Us
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
1/1 ----- 0s 90ms/step

Predictions:
Input: [0 0] -> Predicted: 0
Input: [0 1] -> Predicted: 1
Input: [1 0] -> Predicted: 1
Input: [1 1] -> Predicted: 0

```

✓ Lab 6 || 26 Feb

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_
from sklearn.feature_extraction.text import TfidfVectorizer

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam

df = pd.read_csv('/spam_or_not_spam.csv')
X_text = df['email'].fillna('')
y = df['label']

vectorizer = TfidfVectorizer(max_features=5000)
X = vectorizer.fit_transform(X_text).toarray()

```



```

x_train, x_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# =====
# 4. Build Model
# =====
def create_model(lr):
    model = Sequential()
    model.add(Dense(64, activation='relu', input_shape=(X.shape[1],)))
    model.add(Dense(32, activation='relu'))
    model.add(Dense(1, activation='sigmoid'))

    model.compile(
        optimizer=Adam(learning_rate=lr),
        loss='binary_crossentropy',
        metrics=['accuracy']
    )
    return model

# =====
# 5. Training + Graph
# =====
learning_rates = [0.01, 0.001, 0.005]

for lr in learning_rates:

    print("\n=====")
    print("Training with LR =", lr)

    model = create_model(lr)

    history = model.fit(
        X_train, y_train,
        epochs=200,
        batch_size=32,
        validation_split=0.1,
        verbose=0
    )

    plt.figure(figsize=(8,5))
    plt.plot(history.history['accuracy'], label='Train Accuracy')
    plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
    plt.plot(history.history['loss'], label='Train Loss')
    plt.plot(history.history['val_loss'], label='Validation Loss')
    plt.title(f"Training vs Validation (LR={lr})")
    plt.xlabel("Epochs")
    plt.ylabel("Value")
    plt.legend()
    plt.grid(True)

```

```

plt.show()

y_pred = (model.predict(X_test) > 0.5).astype("int32")

# =====
# Metrics
# =====
print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall   :", recall_score(y_test, y_pred))
print("F1 Score :", f1_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# =====
# Confusion Matrix Heatmap
# =====
cm = confusion_matrix(y_test, y_pred);

plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d',
            xticklabels=['Ham', 'Spam'],
            yticklabels=['Ham', 'Spam'])
plt.title(f"Confusion Matrix (LR={lr})")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

```

```

=====
Training with LR = 0.01
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: Us
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```

